**Modul Javascript Basic**

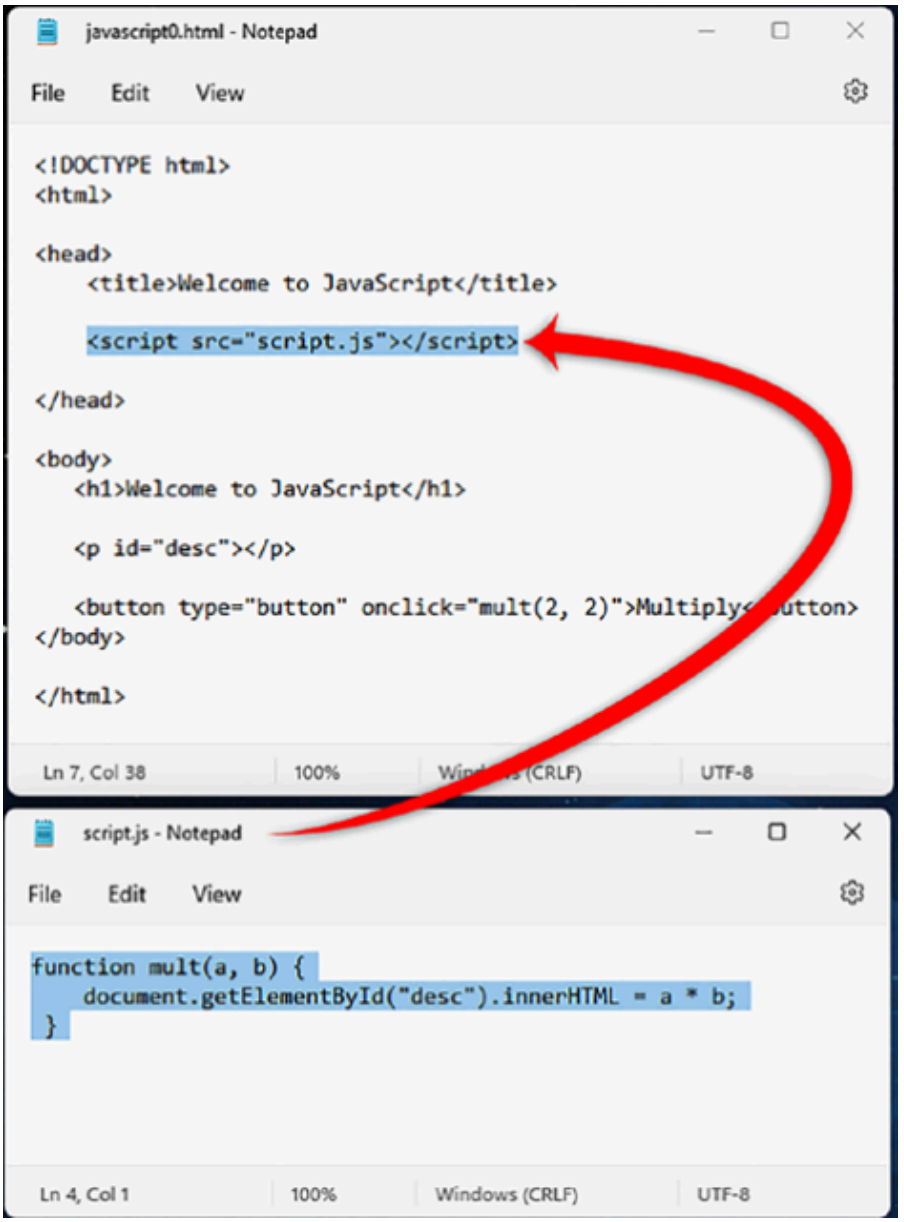
JavaScript is an interpreted, object-based, client-side scripting language that allows you to create dynamic content on a web page. You can animate images, validate data, and create interactive elements. In other words, JavaScript adds behavior to web pages, and it is supported by most modern web browsers such as Chrome, Firefox, Edge, and Safari.

There are three ways you can **add JavaScript to a web page**:

1. You can embed the code between <script> tags within your HTML document. You can add this to the <head> or <body> section of your HTML document. In the example in Figure 8-1, we've added the JavaScript code between the <script> tags in the header.

```
<!DOCTYPE html>
<html>

<head>
    <title>Welcome to JavaScript</title>

    <script>
        function mult(a, b) {
        document.getElementById("desc").innerHTML = a * b;
        }
    </script>

</head>

<body>
   <h1>Welcome to JavaScript</h1>

   <p id="desc"></p>

   <button type="button" onclick="mult(2, 2)">Multiply</button>
</body>

</html>
```

2. You can save the JavaScript code in a separate file and link it to your HTML. This is useful for larger projects where you have multiple HTML documents using the same JavaScript functions.




3. You can also place the JavaScript code directly inside an HTML tag. This is known as inline code.

```
<a href="#" onClick="alert('Welcome !');"> Click Here</a>
```

**Java Script Syntax**

A JavaScript program called a script is a list of statements that contain constructs and commands that perform actions. Let's take a look at some of the basic building blocks of a JavaScript program.

A programming language works in more or less the same way. That is, it, too, has words and rules:

- The words are a set of terms that refer to the specific things that your program works with or the specific ways in which those things can be manipulated. They're known as reserved words or keywords.
- The rules are the ways in which the words can be combined to produce the desired effect. In the programming world, these rules are known as the language's syntax

For example, JavaScript's syntax rules tell you that you can combine the words **`document`** and **`write()`** like so:

**`document.write().`**

If you use

**`write().document`** or **`document write()`**

or any other combination, the page-reading machine won't understand you.

**Understanding Variables**

**Declaring a variable**

| | |
|---|---|
| With `let` | `let VariableName = value;`<br><br>Value may vary through script |
| With `const` | `const VariableName = value;`<br><br>Value will remain constant throughout script |

### Anonymous Functions

Why use anonymous functions? Well, first, you don't have to if you don't want to. Second, the main reason to use anonymous functions is to avoid creating a named object when you don't need to. Every large web project has a huge namespace, which refers to the full collection of identifiers you assign to things like variables and functions. The larger the namespace, the greater the chance of a namespace collision

### Example

```
const preTipTotal = 100.00;
const tipPercentage = 0.15;

function calculateTip(preTip, tipPercent) {
 const tipResult = preTip * tipPercent;
 return (tipResult);
}

const tipCost = calculateTip(preTipTotal, tipPercentage);
const totalBill = preTipTotal + tipCost;
document.write("Your total bill is $" + totalBill);
```

This code defines the named function calculateTip() and later uses the tipCost variable to store the function result. This is a perfect example of when a named function is not needed because you only ever use the named function to calculate the tipCost value. Adding an identity to the namespace when you don't have to is called polluting the namespace, and it's a big no-no in modern JavaScript programming.

You can rewrite this code to use an anonymous function instead

```
const preTipTotal = 100.00;
const tipPercentage = 0.15;

// Declare tipCost using an anonymous function
const tipCost = function (preTip, tipPercent) {
 const tipResult = preTip * tipPercent;
 return (tipResult);
}

const totalBill = preTipTotal + tipCost(preTipTotal, tipPercentage);
document.write("Your total bill is $" + totalBill);
```

As you progress in JavaScript, you'll find yourself using anonymous functions constantly. When you get to that stage, you'll be happy to know that ES6 also offers a simpler anonymous function syntax.

You can rewrite code above using an arrow function:

```
// Declare tipCost using an arrow function
const tipCost = (preTip, tipPercent) => {
 const tipResult = preTip * tipPercent;
 return (tipResult);
}
```

**Manipulating Element**

**Adding an element to the page**

When you add an element, you always specify the parent element to which it will be added, and then you decide whether you want the new element added to the end or to the beginning of the parent's collection of children.

To add an element to the page, you follow three steps:

1. Create an object for the type of element you want to add.

   Use the document object's `createElement()` method:

   ```
   document.createElement(elementName)
   ```

   This method creates the element and then returns it, which means you can store the new element in a variable. Here's an example:

   ```
   const newArticle = createElement("article");
   ```

2. Add the new object from Step 1 as a child element of an existing element.

   With your element created, Step 2 is to add it to an existing parent element. You have four choices:

   a. Append the new element to the end of the parent's collection of child elements: Use the append() method:
   b. Prepend the new element to the beginning of the parent's collection of child elements: Use the prepend() method:
   c. Insert the new element just after an existing child element of the parent: Use the after() method:
   d. Insert the new element just before an existing child element of the parent: Use the before() method:

3. Insert some text and tags into the new object from Step 1.

   With your element created and appended to a parent, the final step is to add some text and tags using the innerHTML property:

   ```
   element.innerHTML = text
   ```

**Inserting text or HTML into an element**

You can use an element's innerHTML property to overwrite that element's tags and text, as I describe in the previous section. However, it's often the case that you want to keep the element's existing tags and text and insert new tags and text. Each element offers a couple of methods that enable you do to do this:

a. To insert just text into an element: Use the **`insertAdjacentText()`** method:

   **`element.insertAdjacentText(location, text)`**

   - **`element`**: A reference to the element into which the new text will be inserted.
   - **`location`**: A string specifying where you want the text inserted. I outline your choices here shortly.
   - **`text`**: A string containing the text you want to insert.

b. To insert tags and text into an element: Use the **`insertAdjacentHTML()`** method:

   **`element.insertAdjacentHTML(location, data)`**

   - **`element`**: A reference to the element into which the new tags and text will be inserted.
   - **`location`**: A string specifying where you want the tags and text inserted. I outline your choices here shortly.
   - **`data`**: A string containing the tags and text you want to insert.

For both methods, you can use one of the following strings for the **`location`** argument:

- "beforebegin": Inserts the data outside of and just before the element
- "afterbegin": Inserts the data inside the element, before the element's first child
- "beforeend": Inserts the data inside the element, after the element's last child
- "afterend": Inserts the data outside of and just after the element

**Modifying CSS with JavaScript**

Although you specify your CSS rules in a static stylesheet (.css) file, that doesn't mean that the rules themselves have to be static. With JavaScript on the job, you can modify an element's CSS in a number of ways.

You can

**a. Changing an element's styles**

Most HTML tags can have a style attribute that you use to set inline styles. Because standard attributes all have corresponding element object properties.

When referencing these style properties, you need to keep two things in mind:

- **For single-word CSS properties** (such as `color` and `visibility`), use all-lowercase letters.
- F**or multiple-word CSS properties**, drop the hyphen and use uppercase for the first letter of the second word and for each subsequent word if the

property has more than two. For example, the `font-size` and `border-left-width` CSS properties become the `fontSize` and `borderLeftWidth` style object properties.

Here's an example:

```
const pageTitle = document.querySelector("h1");
pageTitle.style.fontSize = "64px";
pageTitle.style.color = "maroon";
pageTitle.style.textAlign = "center";
pageTitle.style.border = "1px solid black";
```

This code gets a reference to the page's first `<h1>` element. With that reference in hand, the code then uses the style object to style four properties of the heading: `fontSize, color, text-align`, and `border`

**b. Adding a class to an element**

If you have a class rule defined in your CSS, you can apply that rule to an element by adding the class attribute to the element's tag and setting the value of the class attribute equal to the name of your class rule.

**`element.classList.add(class)`**

- **`element`**: The element you're working with.
- **`class`**: A string representing the name of the class you want to add to element. You can add multiple classes by separating each class name with a comma.

Here's an example:

CSS:

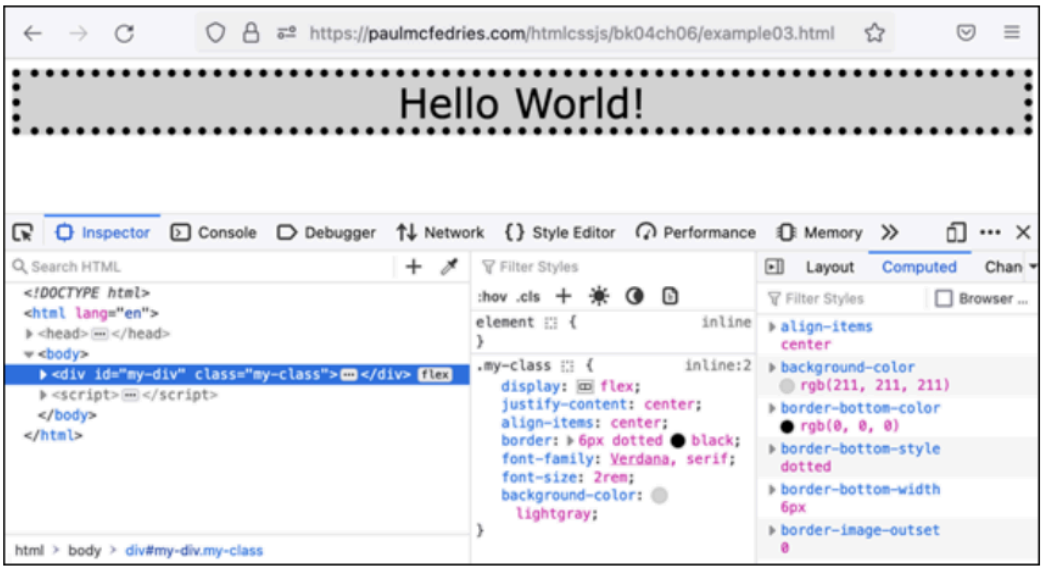
```
.my-class {
 display: flex;
 justify-content: center;
 align-items: center;
 border: 6px dotted black;
 font-family: Verdana, serif;
 font-size: 2rem;
 background-color: lightgray;
}
```

HTML:

```
<div id="my-div">
 Hello World!
</div>
```

Javascript:

```
document.getElementById('my-div').classList.add('my-class');
```




### c. Removing a class

To remove a class from an element's class attribute, the classList object offers the `remove()` method:

```
element.classList.remove(class)
```

- **`element`**: The element you're working with.
- **`class`**: A string representing the name of the class you want to remove from element. You can remove multiple classes by separating each class name with a comma.

Here's an example:

```
document.getElementById('my-div').classList.remove('my-class');
```

### d. Toggling a class

The toggle() method checks the element for the specified class; if the class is there, JavaScript removes it; if the class isn't there, JavaScript adds it. Sweet! Here's the syntax:

```
element.classList.toggle(class)
```

Here's an example:

```
document.getElementById('my-div').classList.toggle('my-class');
```

### e. Reading an attribute value
### f. Setting an attribute value
### g. Removing an attribute
### h. Building Reactive Pages with Events

An event handler consists of two parts:

- Event listener: An instruction to the web browser to watch out ("listen") for a particular event occurring on a particular element.
- Callback function: The code that the web browser executes when it detects that the event has occurred.

**Event types**

1. **Document:** Events that fire in relation to the loading of the document object. The only event you need to worry about here is DOMContentLoaded, which fires when the document object has completed loading.
2. **Mouse:** Events that fire when the user does something with the mouse (or a similar device, such as a trackpad or touchscreen). The most important events in this category are click (the user clicks the mouse); dblclick (the user double-clicks the mouse); and mouseover (the user moves the mouse pointer over an element).
3. **Keyboard:** Events that fire when the user interacts with the keyboard. The main event in this category is keypress, which is fired when the user presses a key.
4. **Form:** Events associated with web page forms. The important ones are focus (an element gains the focus, for example, when the user tabs to a form control); blur (an element loses the focus); change (the user changes the value of a form control); and submit (the user submits the form).
5. **Browser window:** Events that fire when the user interacts with the browser window. The two main events here are scroll, which fires when the user scrolls the window vertically or horizontally, and resize, which fires when the user changes the window width or height.

You configure your code to listen for and react to an event by setting up an event handler using the element object's addEventListener() method. Here's the syntax:

element.addEventListener(event, callback);

- element: The web page element to be monitored for the event. The event is said to be bound to the element.
- event: A string specifying the name of the event you want the browser to listen for. For the main events I mention in the previous section, use one of the following, enclosed in quotation marks: `DOMContentLoaded, click, dblclick, mouseover, keypress, focus, blur, change, submit, scroll, or resize.`
- callback: The callback function that JavaScript executes when the event occurs. The callback can be an anonymous function or a reference to a named function.

Here's an example:

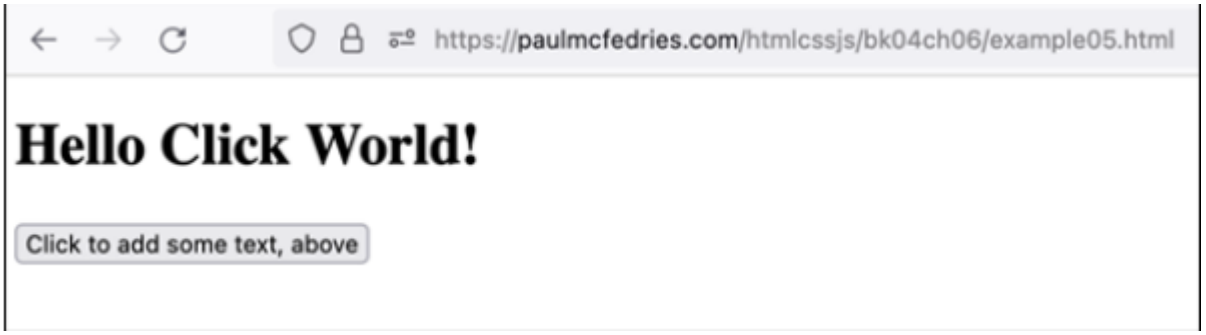
HTML:

```
<div id="my-div"></div>

<button id="my-button">Click to add some text, above</button>
```

Javascript

```
const myButton = document.getElementById('my-button');
myButton.addEventListener('click', function() {
 const myDiv = document.getElementById('my-div');
 myDiv.innerHTML = '<h1>Hello Click World!</h1>';
});
```

The HTML sets up an empty div element and a button element. The JavaScript code attaches a click event listener to the button, and the callback function adds the HTML string <h1>Hello Click World! </h1> to the div. Figure below shows the resulting page after the button has been clicked.




If you want to run some code after the web page document has loaded, add an event handler to the document object that listens for the DOMContentLoaded event

```
document.addEventListener('DOMContentLoaded', function() {
 console.log('We are loaded!');
 });
```

## *Website Development Exercise*

Instructions:

Create a simple website consisting of HTML, CSS, and JavaScript with the following components:

1. Form: Include a form with input fields and a submit button.
2. CSS: Style the website for desktop screens using CSS.
3. JavaScript for Form Data: Use JavaScript to retrieve and display data from the form.

Requirements:

- Design the HTML structure including the form elements.
- Write CSS to style the website for desktop screens.
- Write CSS media queries to adjust the layout and styling for mobile screens.
- Implement JavaScript functions to handle form data retrieval.
- Ensure that the website is responsive and functions correctly on both desktop and mobile devices.

Submission:

Submit the "**NIM_NAMA_Modul3.pdf"** archive containing all files as your assignment.